Architecture de la bibliothèque

Guide de lecture : à quoi sert chaque dossier, à quoi sert chaque fichier

Documentation Architecturale

23 juillet 2026

Table des matières

1 Comment lire ce document **2**

2 Vue d'ensemble : la chaîne de traitement **2**

2.1 Pourquoi séparer `model` de `rendering` ? 2

2.2 Les trois règles qui reviennent partout 2

3 Le dossier racine : `umlg/` **3**

4 `annotation/` — le vocabulaire offert à l'utilisateur **3**

5 `model/` — le diagramme, dit et non dessiné **5**

6 `extraction/` — la lecture du code **6**

7 `rendering/` — le dessin **7**

8 Ajouter une nature de lien : le mode d'emploi **9**

9 `src/test/` — les tests, en miroir **9**

10 Récapitulatif en une page **11**

1 Comment lire ce document

Ce document répond à une seule question, fichier par fichier : *pourquoi celui-là existe-t-il, et que se passerait-il s'il n'existait pas ?* Il ne remplace pas les spécifications des annotations (`annotation.tex`), qui décrivent ce que l'utilisateur *écrit* ; il décrit ce que la bibliothèque *fait* de ce qu'il a écrit.

Le vocabulaire tient en trois mots, employés partout dans le code :

— une **entité** est une classe qui apparaîtra dans le diagramme (une boîte) ;
— un **lien** (ou *relation*) est un trait entre deux boîtes ;
— un **membre** est un attribut ou une méthode, écrit à l'intérieur d'une boîte.

2 Vue d'ensemble : la chaîne de traitement

La bibliothèque transforme du code Java annoté en une image. Ce trajet se fait en quatre étapes, et *chaque dossier correspond exactement à une étape*. C'est la clé de lecture principale : si vous savez à quelle étape appartient un problème, vous savez dans quel dossier chercher.

`annotation/` ce que l'utilisateur écrit dans son code

↓

`extraction/` lecture du code compilé, par réflexion

↓

`model/` le diagramme, décrit sans savoir comment il sera dessiné

↓

`rendering/` traduction en code PlantUML, puis en image

2.1 Pourquoi séparer `model` de `rendering` ?

C'est la séparation la plus importante, et la moins évidente. On pourrait croire qu'un modèle intermédiaire est une complication inutile : pourquoi ne pas écrire directement le code PlantUML pendant qu'on lit les annotations ?

Parce que les deux étapes n'ont pas les mêmes questions à trancher. L'extraction se demande *« que dit ce code ? »* ; le rendu se demande *« comment dessiner cela sans casser la syntaxe de PlantUML ? »*. Mélangées, ces deux questions se contaminent : on se met à refuser un nom de classe parfaitement légitime en Java — `"Compte (bancaire)"` — au motif qu'il gênerait le moteur de rendu. Le défaut serait alors placé au mauvais endroit, et l'utilisateur paierait pour une contrainte qui ne le regarde pas.

Séparés, chacun fait son travail : l'extraction accepte tout texte qui a un sens, et le rendu l'*encode* au moment de l'écrire, en fonction de l'endroit où il atterrit. C'est le même raisonnement qu'entre « filtrer les apostrophes » et « utiliser une requête préparée ».

Bénéfice secondaire, mais réel : le jour où un autre moteur est visé (Mermaid, Graphviz), seul `rendering/` est à remplacer.

2.2 Les trois règles qui reviennent partout

Une fois ces trois règles comprises, la plupart des décisions du code s'expliquent d'elles-mêmes.

1. Les gardes appartiennent au modèle, le contexte appartient à l'extraction. Une règle du type « une multiplicité s'écrit `0..*` » est vérifiée dans `model/`, parce que le modèle doit rester sain quelle que soit la façon dont on le construit. Mais le modèle ne connaît pas votre code : il peut seulement dire *« "0..x" n'est pas une multiplicité valide »*. Sur un projet de quarante classes, ce message ne mène nulle part. L'extraction, elle, sait *où* : elle rattrape l'erreur et y ajoute `com.banque.Compte.solde`.

2. La sûreté syntaxique s'obtient à la sortie, jamais à l'entrée. Aucun caractère n'est interdit dans un nom, un rôle ou une étiquette. Ce sont des textes destinés à être lus par un humain. C'est `PlantUmlEscaper` qui les rend inoffensifs au moment de les écrire. Seul ce dont le vocabulaire est *fermé* — les multiplicités, les couleurs — est vérifié par liste blanche.

3. L'échec est franc, parce que PlantUML ne l'est pas. Face à un nom de classe inconnu, PlantUML n'échoue pas : il invente une boîte vide portant ce nom. Face à une couleur inconnue, il produit une image d'erreur parfaitement valide. Dans les deux cas, le résultat est un fichier de taille normale, indiscernable d'un vrai diagramme. C'est pourquoi la bibliothèque vérifie elle-même, en amont, et refuse bruyamment.

3 Le dossier racine : `umlg/`

Un seul fichier, et c'est voulu : c'est la porte d'entrée.

Fichier	Rôle
`UMLGenerator`	La façade publique. C'est la seule classe qu'un utilisateur de la bibliothèque a besoin de connaître. Elle enchaîne les deux grandes étapes — scanner, puis écrire — et rien d'autre. Elle se construit par un *builder* (`sourceDirectory`, `outputFile`), ce qui permet de vérifier tôt que le dossier existe et que le format demandé est bien du `.png` ou du `.svg`. Contient aussi un `main`, pour un usage en ligne de commande.

4 `annotation/` — le vocabulaire offert à l'utilisateur

Ce dossier est, avec `UMLGenerator`, la **surface publique** de la bibliothèque : tout le reste est de la mécanique interne. Les fichiers y sont très courts, car une annotation Java ne fait rien — elle se contente de *déclarer*. Aucune validation n'y a lieu : le compilateur garantit les types, le modèle garantit le reste.

Fichier	Rôle
`UMLClass`	Le déclencheur. Une classe sans cette annotation n'existe pas pour la bibliothèque. Porte les quatre réglages de la boîte : `name` (l'étiquette affichée), `color`, `stereotype`, et `strategy` (le seuil de visibilité, voir ci-dessous).
`UMLVisibility`	L'énumération des seuils : `NONE` (aucun membre affiché, valeur par défaut), `PUBLIC`, `PROTECTED`, `PACKAGE`, `PRIVATE` (tout). Répond à la question « jusqu'à quel niveau de visibilité Java les membres apparaissent-ils ? ».
`UMLInclude` `UMLExclude`	Les deux dérogations, posées sur un attribut ou une méthode. Elles l'emportent toujours sur la stratégie : `@UMLInclude` fait apparaître ce qu'elle masquerait, `@UMLExclude` masque ce qu'elle afficherait. En cas de conflit, `@UMLExclude` gagne.
`UMLAssociation`	Posée sur un *attribut*, elle le transforme en trait entre deux boîtes. Tous ses paramètres sont facultatifs : cardinalités, rôles, qualificatif, étiquette, sens du trait, et `target` pour désigner explicitement la classe visée quand elle ne peut pas se déduire.
`UMLRelationType`	La sémantique du trait : `ASSOCIATION` (lien simple), `AGGREGATION` (losange creux), `COMPOSITION` (losange plein). Le code Java ne distingue pas ces trois cas — `private List<Ligne> lignes` peut être l'un ou l'autre — seul le métier tranche.
`UMLDirection`	Une contrainte de *placement* : `UP`, `DOWN`, `LEFT`, `RIGHT`, ou `NONE`. Ne change rien au sens du diagramme, seulement à sa mise en page.
`UMLDependency`	Posée sur la *classe*, elle déclare « celle-ci a besoin de celle-là » sans la détenir. C'est la seule façon de faire apparaître un lien que le code ne porte pas structurellement : un service appelé dans le corps d'une méthode n'est pas un attribut, rien ne pourrait en être déduit.
`UMLDependencies`	Purement technique, jamais écrite à la main. Java exige ce conteneur pour qu'une annotation puisse être *répétée* sur la même classe. C'est ce qui permet d'écrire plusieurs `@UMLDependency` d'affilée.
`package-info`	Documentation du paquetage : son statut public, et la marche à suivre pour l'étendre.

5 `model/` — le diagramme, dit et non dessiné

Ce dossier décrit le diagramme *sans savoir* qu'il sera rendu par PlantUML. Tous les fichiers y sont des `record` Java : des objets immuables, qui vérifient leur cohérence à la construction et ne peuvent plus changer ensuite. Un modèle qui se laisserait modifier après coup ne pourrait rien garantir.

Fichier	Rôle
`UmlProject`	L'ensemble des entités du diagramme. Refuse d'être vide : scanner un projet où personne n'a annoté doit produire une erreur compréhensible, pas une image blanche.
`UmlEntity`	Une boîte. Porte deux noms qu'il ne faut surtout pas confondre : `qualifiedName` est son *identité* (`com.banque.Compte`), `name` son *étiquette* (ce que le lecteur voit). L'étiquette est libre, donc ni unique ni reliée au code ; c'est pourquoi les liens désignent toujours leur cible par le nom qualifié, qui, lui, ne ment pas. Contient aussi ses membres et *tous* ses liens sortants.
`UmlField` `UmlMethod`	Une ligne de la boîte : nom, type, symbole de visibilité (`+ - # ~`), caractère statique, et pour la méthode la liste des types de paramètres.
`UmlRelation`	**La pièce centrale pour la suite du projet.** C'est une interface *scellée* : elle énumère elle-même les natures de liens qui existent. Grâce à elle, une entité n'a qu'*une seule* liste de liens, toutes natures confondues. Voir la section 8, qui explique ce que cela change concrètement.
`UmlAssociation`	Un lien *structurel* : une extrémité détient l'autre. Porte les cardinalités, les rôles, le qualificatif, l'étiquette et le sens. Vérifie ici que les multiplicités sont bien formées *et* cohérentes : `5..2` est syntaxiquement irréprochable et vide de sens, PlantUML l'afficherait sans broncher.
`UmlDependency`	Un lien d'*usage* : la source a besoin de la cible, sans la détenir. D'où l'absence de cardinalité et de rôle — il n'y a pas d'extrémité à décrire, seulement un usage à signaler.
`package-info`	Documentation du paquetage : où sont les gardes, et pourquoi tout y est immuable.

Une remarque sur l'absence de « source » dans les liens. Ni `UmlAssociation` ni `UmlDependency` ne mentionnent la classe d'où part le trait. Ce n'est pas un oubli : la source *est* l'entité dans la liste de laquelle le lien figure. Un lien ne peut donc être ni orphelin, ni attribué à une source autre que celle qui le déclare — deux erreurs qu'il serait

autrement possible de commettre.

6 `extraction/` — la lecture du code

Ce dossier lit les fichiers `.class` compilés et les traduit en modèle, par *réflexion* (l'API Java qui permet d'interroger une classe à l'exécution).

Il compte onze fichiers là où il n'y en avait que deux, et cette multiplication est le cœur de la réorganisation. La raison est simple : chacun de ces sujets **grossit séparément**. Le catalogue des conventions reconnues comme de la plomberie n'a aucune raison de faire grossir la déduction des types génériques, ni l'inverse. Réunis dans un seul fichier, ils se gênaient.

Seuls `ProjectScanner` et `EntityExtractor` sont publics ; tous les autres sont volontairement invisibles hors du paquetage. C'est là que se lit le contrat : ce qui n'est pas public n'a pas à rester stable.

Fichier	Rôle
`ProjectScanner`	Parcourt un dossier de `.class`, les charge, et retient ceux qui portent `@UMLClass`. Traduit au passage le chemin du fichier en nom de classe. Si un fichier est illisible, l'erreur *nomme* la classe fautive.
`EntityExtractor`	Le chef d'orchestre, et rien d'autre : 63 lignes. Il valide l'en-tête, demande les membres, puis parcourt son **catalogue d'extracteurs de liens**. Il ne connaît plus aucune nature de lien en particulier — seulement leur liste.
`ClassHeaderValidator`	Les gardes sur ce que `@UMLClass` déclare : `name`, `stereotype`, `color`. Illustre la règle n°2 : le nom n'a qu'une contrainte (désigner quelque chose), la couleur est vérifiée par liste blanche.
`VisibilityPolicy`	Répond à « ce membre paraît-il ? », et « sous quel symbole ? ». Applique d'abord les dérogations, puis le seuil. Écrit avec un `switch` exhaustif : ajouter une valeur à `UMLVisibility` sans dire ce qu'elle retient devient une erreur de compilation.
`NoiseFilter`	Reconnaît la *plomberie* : getters, setters, `toString`, accesseurs de `record`, méthodes que `javac` ajoute aux `enum`. Détail crucial : la détection exige la convention JavaBean *complète* (préfixe suivi d'une majuscule, arité, type de retour). Un simple test de préfixe faisait disparaître silencieusement `setup()`, `issue()`, `getaway()`, `isoler()` — de vraies méthodes métier.
`MemberExtractor`	Parcourt attributs et méthodes, et traduit ceux qui sont retenus. Écarte au passage un attribut promu en lien : en UML, une propriété se note dans la boîte *ou* comme un trait, jamais les deux.

Fichier	Rôle
`TargetResolver`	Répond à la question la plus difficile du dossier : *vers quoi pointe ce lien quand l'annotation ne le dit pas ?* Sait déduire l'élément d'un tableau, le contenu d'une collection ou d'un `Optional`, la valeur d'une `Map`, la borne haute d'un joker. Refuse franchement quand le code ne dit rien (`List` brute, `List<T>`) : deviner produirait un lien faux, qui se lit exactement comme un lien vrai.
`RelationExtractor`	**Le point d'extension.** Une interface : « voici une classe, donne-moi les liens que tu sais y lire ». Une implémentation par annotation de lien.
`AssociationExtractor`	Lit les `@UMLAssociation` posées sur les attributs. Note qu'un lien n'est *pas* soumis à la stratégie de visibilité : une composition ne disparaît pas parce que le champ qui la porte est privé — c'est même le cas le plus courant.
`DependencyExtractor`	Lit les `@UMLDependency` posées sur la classe. Écarte deux fautes silencieuses : une classe qui dépend d'elle-même (une boucle sans information), et deux déclarations identiques (deux traits superposés, invisibles à l'œil).
`Emplacement`	Une seule méthode, et pourtant un fichier à elle : elle produit le « où » des messages d'erreur (`com.banque.Compte.solde`). Chaque extracteur à venir doit désigner la déclaration fautive de la même façon, et non chacun à la sienne.
`package-info`	Documentation du paquetage : statut de chaque classe, et le point d'extension.

7 `rendering/` — le dessin

Ce dossier est le seul à connaître la syntaxe de PlantUML. Seul `PlantUmlWriter` est public.

Fichier	Rôle
`PlantUmlWriter`	Le chef d'orchestre du rendu : construit le document, puis demande à PlantUML de produire le `.svg` ou le `.png`. Applique la règle des **deux passes** — toutes les boîtes d'abord, tous les traits ensuite — car un trait nomme ses extrémités par un identifiant qui n'existe qu'une fois la boîte déclarée.

Fichier	Rôle
`PlantUmlSyntaxBuilder`	Assemble le document. Ne connaît plus de la syntaxe que l'*ordre* des morceaux, ce qu'aucun de ses collaborateurs ne peut savoir seul : les classes avant les liens, et le bloc de style en tête bien qu'il ne soit calculable qu'à la fin.
`AliasRegistry`	L'annuaire du document : à chaque classe, l'identifiant court par lequel PlantUML la désignera. Traite deux pièges que PlantUML ne signale pas : un nom sans aucun caractère alphanumérique ASCII (un nom en idéogrammes, ou « ... ») donnerait un identifiant vide, et deux noms distincts peuvent se réduire au même (« Mon.Client » et « MonClient »), auquel cas PlantUML fusionnerait silencieusement les deux classes.
`ClassBoxRenderer`	Écrit une boîte : son en-tête, ses attributs, le filet séparateur, ses méthodes.
`RelationRenderer`	Écrit un trait, quelle que soit sa nature, par un `switch` sur `UmlRelation`. Retient au passage la largeur de la plus large étiquette d'extrémité : c'est ici, et nulle part ailleurs, que le diagramme apprend de quel espace il aura besoin entre ses boîtes.
`PlantUmlEscaper`	L'encodage de sortie (règle n°2). Neutralise ce qui casserait la syntaxe ou permettrait une injection. Ce n'est pas cosmétique : les étiquettes PlantUML acceptent des balises, dont `<img:/chemin>`, qui fait lire un fichier local au moteur de rendu.
`ColorCodec`	Vérifie qu'une couleur existe vraiment, en interrogeant le catalogue de PlantUML — la seule autorité en la matière, et la raison pour laquelle cette vérification n'est pas à l'extraction.
`PlantUmlLayout`	Tout ce qui ne change rien à ce que le diagramme *dit*, et tout à la facilité avec laquelle on le *lit* : police, couleurs, coins arrondis, et surtout l'espacement entre boîtes, calculé d'après la plus longue étiquette — c'est ce réglage qui décide, à lui seul, si les textes se chevauchent.
`package-info`	Documentation du paquetage, dont l'asymétrie « ouvert à l'extraction, fermé au rendu ».

8 Ajouter une nature de lien : le mode d'emploi

C'est le scénario pour lequel l'architecture a été réorganisée. Supposons que vous vouliez ajouter `@UMLInheritance` (l'héritage). Voici la liste complète de ce qu'il faut faire :

1. **Déclarer l'annotation** : un nouveau fichier dans `annotation/`.
2. **Déclarer le lien dans le modèle** : un nouveau `record` dans `model/`, qui implémente `UmlRelation`, et une mention dans la clause `permits` de celle-ci.
3. **Écrire la lecture** : une classe dans `extraction/` qui implémente `RelationExtractor`, plus *une ligne* dans le catalogue `EXTRACTEURS` d'`EntityExtractor`.
4. **Écrire le dessin** : deux `case` dans `RelationRenderer`.

Ce qui ne bouge pas. Ni `UmlEntity`, ni le corps d'`EntityExtractor`, ni `PlantUmlSyntaxBuilder`, ni `PlantUmlWriter`. Auparavant, `UmlEntity` devait être modifié en cinq endroits, et `PlantUmlSyntaxBuilder` en deux.

Ce que le compilateur garantit. Si vous oubliez l'étape 4, le projet **ne compile pas**. Le compilateur pointe précisément les deux `switch` à compléter. C'est toute la raison d'être de l'interface *scellée* : auparavant, un lien non dessiné disparaissait simplement du diagramme, sans erreur, sans avertissement — le dessin paraissait juste un peu plus simple qu'il n'aurait dû.

Pourquoi ouvert d'un côté et fermé de l'autre ? L'asymétrie est délibérée, et repose sur ce que coûte l'oubli. Oublier un extracteur ne produit *rien* : la liste est explicite et se relit. Oublier un dessin produit un diagramme *plausible* mais amputé d'une famille entière de traits — l'erreur la plus coûteuse, puisque rien ne la signale.

9 `src/test/` — les tests, en miroir

Les tests suivent exactement le découpage des sources : à chaque classe qui porte une responsabilité correspond un fichier de tests qui la vérifie. Cela vaut mode d'emploi : pour comprendre ce que fait `NoiseFilter`, `NoiseFilterTest` est souvent plus instructif que le code lui-même, parce qu'il montre les cas limites qui ont motivé chaque ligne.

Fichier	Ce qu'il vérifie
Racine	
`UMLGeneratorTest`	La façade : dossier inexistant, format de sortie refusé, valeurs par défaut.
`UMLGeneratorIntegrationTest`	La chaîne complète, de bout en bout, jusqu'au fichier image produit.
`model/` — les invariants, sans réflexion	
`UmlModelTest`	Projet, entité, méthode, attribut : nullité, copies défensives, égalité.
`UmlAssociationTest`	Multiplicités (vocabulaire fermé) et textes libres.
`UmlDependencyTest`	Étiquettes et cible.

Fichier	Ce qu'il vérifie
`UmlEntityRelationsTest`	La liste unique de liens et ses vues filtrées.
`extraction/` — une responsabilité, un fichier	
`EntityExtractorTest`	L'ordonnancement seul : métadonnées, et le catalogue appelle bien tous ses extracteurs.
`ClassHeaderValidatorTest`	Nom, stéréotype, couleur.
`VisibilityPolicyTest`	Les cinq stratégies et les deux dérogations.
`NoiseFilterTest`	Le filtre anti-bruit, et surtout ses faux positifs.
`MemberExtractorTest`	`enum`, `record`, interfaces, héritage, surcharges, membres générés.
`TargetResolverTest`	La déduction de cible, et les refus quand elle est indécidable.
`AssociationExtractorTest`	Les paramètres de l'annotation, et l'articulation avec la boîte.
`DependencyExtractorTest`	Les dépendances, et les fautes silencieuses écartées.
`ProjectScannerTest`	Le scan, sur le dossier de compilation du projet lui-même.
`ProjectScannerIntegrationTest`	Le scan, sur des `.class` compilés à la volée.
`rendering/`	
`PlantUmlAssert`	Outillage partagé : des assertions sur le code PlantUML produit.
`AliasRegistryTest`	L'annuaire des identifiants, et ses deux pièges.
`PlantUmlSyntaxBuilderTest`	L'assemblage du document.
`PlantUmlSyntaxStructureTest`	La structure produite : boîtes, compartiments, en-tête.
`PlantUmlAssociationTest`	Le rendu des liens structurels.
`PlantUmlDependencyTest`	Le rendu des liens d'usage.
`PlantUmlEscapeTest`	L'encodage de sortie — le fichier le plus fourni du projet.
`PlantUmlLisibiliteTest`	Les réglages de mise en page.
`PlantUmlWriterTest`	L'écriture du fichier.
`PlantUmlWriterOutputTest`	Les formats de sortie et leur détection.

Pourquoi les tests de rendu ne portent pas les noms des nouvelles classes. Ils sont nommés d'après la *fonctionnalité rendue* (l'encodage, les associations, la mise en page) et non d'après la classe qui l'implémente. C'est un axe de découpage tout aussi légitime, et il a un avantage propre : ces tests vérifient le document produit dans son ensemble, ce qu'aucun test d'une classe isolée ne pourrait faire. Seul `PlantUmlAliasTest` a été renommé `AliasRegistryTest`, l'annuaire étant devenu une classe à part entière.

10 Récapitulatif en une page

Dossier	En une phrase
`umlg/`	La porte d'entrée : une seule classe, `UMLGenerator`.
`annotation/`	Ce que l'utilisateur écrit. Ne valide rien, ne connaît aucun moteur de rendu.
`model/`	Ce que le diagramme dit. Immuable, se garde lui-même, ignore PlantUML.
`extraction/`	Comment on lit le code. Une responsabilité par fichier, parce que chacune grossit de son côté.
`rendering/`	Comment on dessine. Seul à connaître PlantUML, seul à encoder les textes.

Si vous ne deviez retenir qu'une chose : les liens de toutes natures vivent dans une seule liste, décrite par l'interface scellée `UmlRelation`. C'est elle qui fait qu'une annotation nouvelle s'ajoute sans toucher au cœur, et qu'un lien oublié au dessin empêche la compilation au lieu de disparaître discrètement du diagramme.